

FinOps for LLM Spend

A LiteLLM + Prometheus + Grafana stack that tracks cost, tokens, latency, and error rate per model tier — the same cost-attribution playbook used for cloud spend, applied to LLM API spend instead.

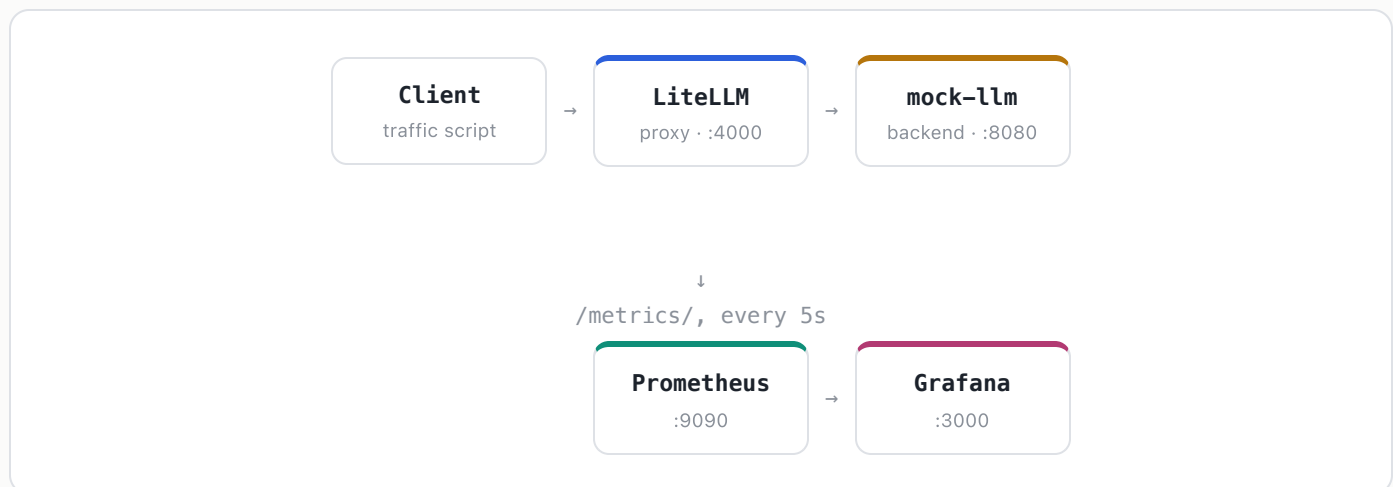
Megha Shivhare · github.com/MeghaShivhare/llm-cost-observability

01 Why this exists

Most LLM cost/observability content comes from ML engineers. This project takes the infra angle: LLM API spend is untracked in most orgs the same way cloud spend was before tagging discipline existed — nobody notices which model or team is burning budget until the bill arrives. This reuses an existing skill set (Prometheus/Grafana, cost-attribution thinking) and points it at LLM spend instead of cloud spend.

02 Architecture

One request's full round trip through the stack:



mock-llm is a minimal OpenAI-compatible endpoint standing in for a real provider, so the stack runs with zero API keys or real spend. **LiteLLM** routes the request, retries transparently on failure, and prices the response from configured per-token rates. **Prometheus** scrapes LiteLLM's metrics every 5 seconds. **Grafana** queries Prometheus and renders the dashboard.

WHY A MOCK BACKEND

Nothing downstream of LiteLLM cares what's actually behind it. Swapping `mock-llm` for a real provider is a one-line change to `api_base` — nothing else in the pipeline changes.

03 Why LiteLLM

Without it there are two worse options: hand-write provider-specific routing, auth, and pricing logic yourself and maintain it as prices change — or pay for a hosted LLM-observability SaaS that routes your data through a third party. LiteLLM gives one request/response shape regardless of the backend, cost math from a one-time price list, Prometheus metrics from one config line, and automatic retries — self-hosted and free.

04 The cost mechanic

`mini-model` and `flagship-model` both route to the identical mock backend, but carry different configured pricing (modeled on real GPT-4o-mini vs GPT-4o rates):

TIER	INPUT \$/TOKEN	OUTPUT \$/TOKEN
mini-model	0.00000015	0.0000006
flagship-model	0.0000025	0.00001

THE KEY MECHANIC

LiteLLM prices a response using the pricing attached to the **requested model alias** — not whatever backend actually served it. That's what produces a ~17x cost difference against a single physical backend, and it's the same mechanic that lets a production alias point at whatever real backend you want without callers knowing.

05 Dashboard — 8 panels

PANEL	QUERY
Total Spend	<code>sum(litellm_spend_metric_total)</code>
Total Requests	<code>sum(litellm_requests_metric_total)</code>
Total Tokens	<code>sum(litellm_total_tokens_metric_total)</code>
p95 API Latency	<code>histogram_quantile(0.95, sum by (le) (rate(litellm_llm_api_latency_metric_bucket[5m])))</code>
Spend by Model Tier	<code>sum by (requested_model) (increase(litellm_spend_metric_total[5m]))</code>
Tokens by Model Tier	<code>sum by (requested_model) (increase(litellm_input_tokens_metric_total[5m]))</code>
Request Rate by Status	<code>sum by (status_code) (rate(litellm_proxy_total_requests_metric_total[5m]))</code>
Deployment Failures	<code>sum by (requested_model) (litellm_deployment_total_requests_total) - sum by (requested_model) (litellm_deployment_success_responses_total)</code>

Every query was verified directly against Prometheus's `/api/v1/query` API with real traffic before being trusted in a panel.

06 Real bugs hit while building this

1 LiteLLM silent for ~60–90s on every start

No log output during startup made it look hung. `docker stats` showed non-zero CPU the whole time — it was loading its internal pricing map, not crash-looping.

2 Prometheus target down, HTTP 401

`/metrics` requires bearer auth by default; Prometheus's scrape config sends none. Fixed with `require_auth_for_metrics_endpoint: false`, acceptable since the endpoint isn't exposed outside the Docker network.

3 Scrape still failing after the auth fix

`/metrics` 307-redirects to `/metrics/`, and Prometheus doesn't follow redirects on scrape. Pointed `metrics_path` at the resolved path directly.

4 A PromQL bug dropped a label mid-query

Subtracting two metrics with `on (model_id)` matched correctly but discarded `requested_model` from the result. Fixed by aggregating each side to `requested_model` before subtracting.

5 Grafana: "data source not found" after a config change

`docker-compose restart` reuses the same container filesystem, so a datasource UID change collided with the old provisioned record. Fixed with a full container recreate.

6 p95 latency read ~9s against a 0.2–1.2s backend

A single Uvicorn worker meant concurrent requests queued behind each other, and queueing delay got counted as latency. Fixed by running 4 workers.

07 How I'd productionize this

- **Real backend** — swap the mock for OpenAI/Anthropic/Bedrock, one config line.
 - **Long-term storage** — Prometheus alone isn't durable; add remote-write to Thanos or Mimir.
 - **Alerting** — LiteLLM already exposes a remaining-budget metric; wire spend-rate and latency-SLO alerts through Alertmanager.
 - **Chargeback** — metrics already carry team/user/api-key labels; map directly to per-team cost breakdowns.
 - **Metrics auth** — replace the disabled-auth shortcut with a scoped credential or network-level isolation.
-